

Documentación Técnica: Control de Versiones con Git y GitHub

Parte 1. Conceptos Básicos

1. ¿Qué es Git?

Es un sistema de control de versiones distribuido que funciona en la computadora local. Permite guardar un historial de cambios en archivos de texto, código o documentación, facilitando volver a versiones anteriores si ocurre algún error.

2. ¿Qué es GitHub?

Es una plataforma web en la nube que aloja repositorios Git. Permite almacenar proyectos de forma remota, colaborar en equipo y construir un portafolio profesional accesible desde cualquier lugar.

3. ¿Cuál es la diferencia entre Git y GitHub?

Git es la herramienta/software de consola de comandos local que administra el historial de cambios. GitHub es la plataforma en línea que almacena dichos repositorios para compartirlos en Internet.

4. ¿Qué es un repositorio?

Es la carpeta principal de un proyecto administrada por Git, donde se guardan todos sus archivos, el historial completo de modificaciones y las distintas ramas de desarrollo.

5. ¿Qué es un commit?

Es un punto de guardado o instantánea del estado de los archivos en un momento específico. Cada commit contiene un identificador único (hash) y un mensaje descriptivo de los cambios realizados.

6. ¿Para qué sirve un archivo README.md?

Es el archivo de presentación de un repositorio. Escrito en formato Markdown, explica la finalidad del proyecto, sus características principales y cómo utilizarlo o contribuir en él.

7. ¿Qué es una rama (branch) y para qué sirve?

Es una línea de trabajo independiente dentro del repositorio. Permite implementar nuevas funciones o correcciones sin alterar la rama principal (main).

Parte 2. Creación del Repositorio en GitHub

Para iniciar el proyecto, primero se crea el repositorio remoto directamente en la plataforma web de GitHub:

1. Iniciar sesión en GitHub: Accede a tu cuenta en <https://github.com> e ingresa a la sección de repositorios.

2. Crear un nuevo repositorio: Haz clic en el botón 'New' (Nuevo repositorio).

3. Configurar los detalles del repositorio:

- Nombre del repositorio: Proyecto_Integrador
- Propietario / Usuario: Tetlahchiwih
- Visibilidad: Pública
- Opciones de inicialización: NO marcar la creación automática de README.md, .gitignore ni licencia (se debe crear completamente vacío).

4. Obtener las instrucciones iniciales de GitHub: Una vez creado el repositorio, GitHub mostrará la pantalla de bienvenida con las instrucciones para inicializar un repositorio local desde la consola de comandos usando la URL remota:

https://github.com/Tetlahchiwih/proyecto_integrador.git

Parte 3. Verificación de Instalación y Configuración Local

1. Verificación de la versión de Git

Abre tu terminal (Git Bash, PowerShell o CMD) y comprueba que Git esté instalado correctamente:

```
git --version
# Comprobación esperada:
# git version 2.45.2.windows.1
```

2. Configuración global del usuario

Establece la identidad del autor para registrar los cambios en la terminal:

```
git config --global user.name "Tetlahchiwih"
git config --global user.email "anagabrielacastillo5@gmail.com"
```

3. Verificación de la configuración

```
git config --global --list
# Comprobación esperada:
# user.name=Tetlahchiwih
# user.email=anagabrielacastillo5@gmail.com
```

Parte 4. Inicialización Local y Sincronización con los Comandos de GitHub

A continuación, se crea la carpeta del proyecto en la computadora local y se aplican exactamente los comandos proporcionados por la interfaz de GitHub para subir el primer commit:

1. Crear la carpeta local e ingresar a ella:

```
cd Desktop
mkdir proyecto_integrador
cd proyecto_integrador
```

2. Inicializar el repositorio local de Git:

```
git init
# Comprobación esperada:
# Initialized empty Git repository in
C:/Users/Jessica/Desktop/Proyecto_Integrador/.git/
```

3. Crear el archivo README.md inicial:

```
echo "# proyecto_integrador" >> README.md
```

4. Ejecutar la secuencia de comandos proporcionada por GitHub:

```
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/Tetlahchiwih/proyecto_integrador.git
git push -u origin main
```

5. Verificación de la vinculación remota:

```
git remote -v
# Comprobación esperada:
# origin https://github.com/Tetlahchiwih/proyecto_integrador.git (fetch)
# origin https://github.com/Tetlahchiwih/proyecto_integrador.git (push)
```

Parte 5. Flujo de Trabajo con Ramas (Branches)

Para mantener organizada la documentación e implementar nuevas mejoras sin alterar la rama principal directamente, se crea una rama secundaria y luego se fusiona.

1. Crear y cambiarse a una rama secundaria:

```
git branch Tarea_1
git checkout Tarea_1
```

2. Crear una carpeta para la Tarea 1 en la nueva rama:

```
mkdir Tarea-1
```

3. Agregar los archivos a través del explorador de archivos

4. Subir la nueva rama e historial actualizado a GitHub:

```
git add .
git commit -m "Adjunto carpeta de Tarea 1"
```

5. Subir la nueva rama e historial actualizado a GitHub:

```
git push origin Tarea_1
```

Parte 6. Reflexión Final

¿Qué parte del proceso se te hizo más fácil?

La ejecución de la secuencia de comandos proporcionada por GitHub en la Parte 4, ya que permitió vincular la carpeta local con la nube de forma directa y clara en un solo paso.

¿Qué error o dificultad tuviste?

Al crear la carpeta nueva e ingresar con el bash, detectamos que ya existía una rama main en esta, por lo que tuvimos que verificar los archivos ocultos de su escritorio, percatándonos de esta forma que efectivamente existía un archivo “.git”, el cual tuvimos que eliminar antes de iniciar el proceso. Además al momento de ejecutar 'git push -u origin main', no pudimos subir la carpeta vacía la primera vez que lo intentamos.

¿Cómo resolverías un problema si git push no funciona?

- 1) Validar que la URL remota esté correcta usando 'git remote -v'.
- 2) Comprobar con 'git status' que los cambios hayan sido guardados con 'git commit'.
- 3) Ejecutar 'git pull origin main' si existen cambios remotos que no se han descargado previamente a la máquina local.
- 4) Revisar los permisos de la cuenta 'Tetlahchiwih' en el repositorio 'Proyecto_Integrador'.

¿Cómo podrías usar GitHub como portafolio profesional?

Subiendo repositorios organizados con descripciones detalladas en el README.md, diagramas de flujo y un historial de commits limpio. Esto permite a los evaluadores comprobar el dominio práctico en Git, la organización del código y el manejo de flujos de trabajo profesionales.

¿Cómo se relaciona Git con soporte TI?

En soporte técnico e infraestructura, Git se utiliza para gestionar el control de versiones de scripts de automatización (Python, Bash, PowerShell), manuales técnicos y configuraciones de red. Facilita la auditoría de cambios y permite restaurar configuraciones estables de manera inmediata si surge algún fallo.